

Appendix: R Code

```
# Import the necessary libraries
library(boot)
library(ggplot2)
library(gridExtra)
library(MASS)
if (!require("MVN")) {
  install.packages("MVN")
}
```

```
## Loading required package: MVN
```

```
## Warning: package 'MVN' was built under R version 4.3.3
```

```
library(MVN)
```

1 Load the data and extract the variables

```
# Load the data
load("1854554.RData")

# Tell R that treatment is categorical
dat$treatment <- as.factor(dat$treatment)
# Add the pulse difference to the data frame
dat$pulse_diff = dat$stimulated_pulse - dat$rest_pulse

# Attach the data frame
attach(dat)

# View the data
# View(dat)
```

2 Exploratory analysis of the data

```
# Print the first few rows of the data
head(dat)

##   rest_pulse stimulated_pulse  bmi treatment pulse_diff
## 1         58              62 23.2         0          4
## 2         67              71 22.2         1          4
## 3         68              75 20.7         0          7
## 4         66              68 22.0         1          2
## 5         61              64 23.7         1          3
## 6         66              73 21.8         1          7

# Summary of the data
summary(dat)
```

```
##      rest_pulse      stimulated_pulse      bmi      treatment      pulse_diff
##  Min.   :58.00    Min.   :60.00    Min.   :18.20    0: 56      Min.   : 1.00
##  1st Qu.:61.00    1st Qu.:67.00    1st Qu.:21.30    1:118     1st Qu.: 4.00
##  Median :64.00    Median :72.00    Median :21.90           Median : 7.00
##  Mean   :64.91    Mean   :72.49    Mean   :21.90           Mean   : 7.58
##  3rd Qu.:67.00    3rd Qu.:76.00    3rd Qu.:22.48           3rd Qu.:10.00
##  Max.   :80.00    Max.   :98.00    Max.   :25.80           Max.   :23.00
```

```
par(mfrow = c(1, 3))
```

```
# Box plot for BMI by Treatment
```

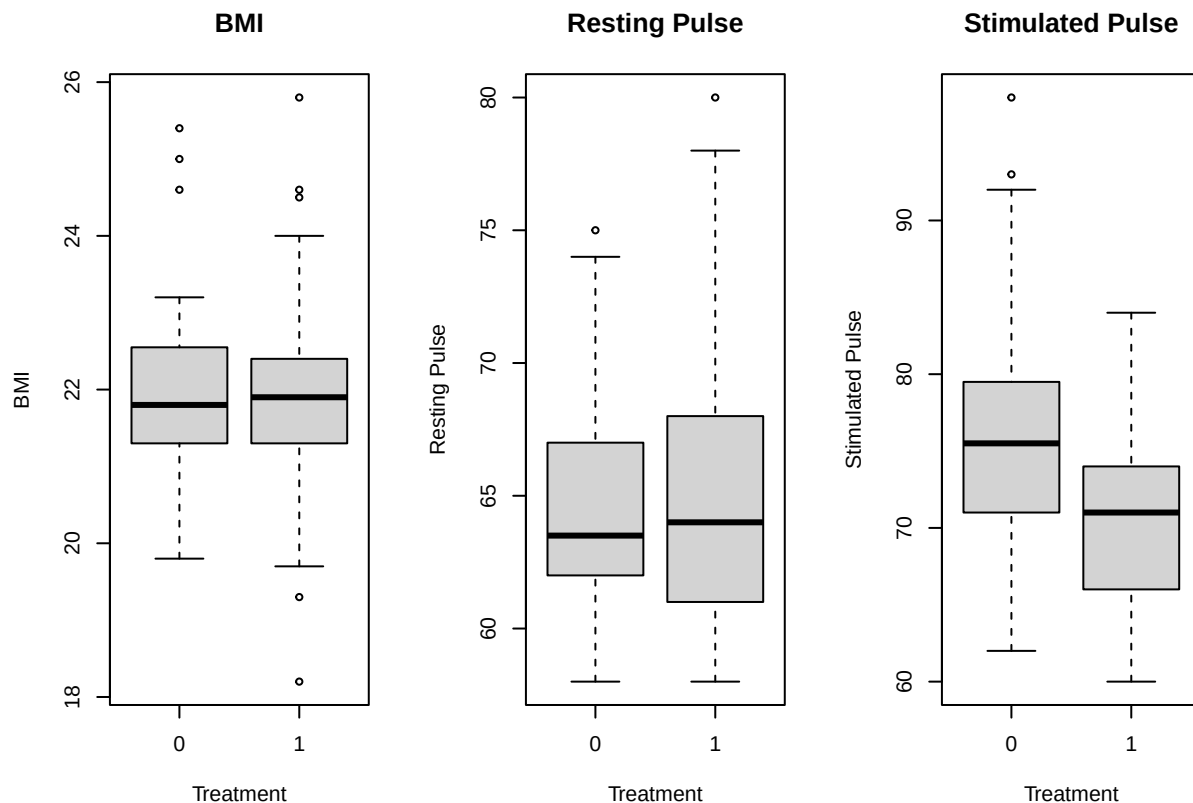
```
boxplot(bmi ~ treatment, main = "BMI",
        xlab = "Treatment", ylab = "BMI")
```

```
# Box plot for Resting Pulse by Treatment
```

```
boxplot(rest_pulse ~ treatment, main = "Resting Pulse",
        xlab = "Treatment", ylab = "Resting Pulse")
```

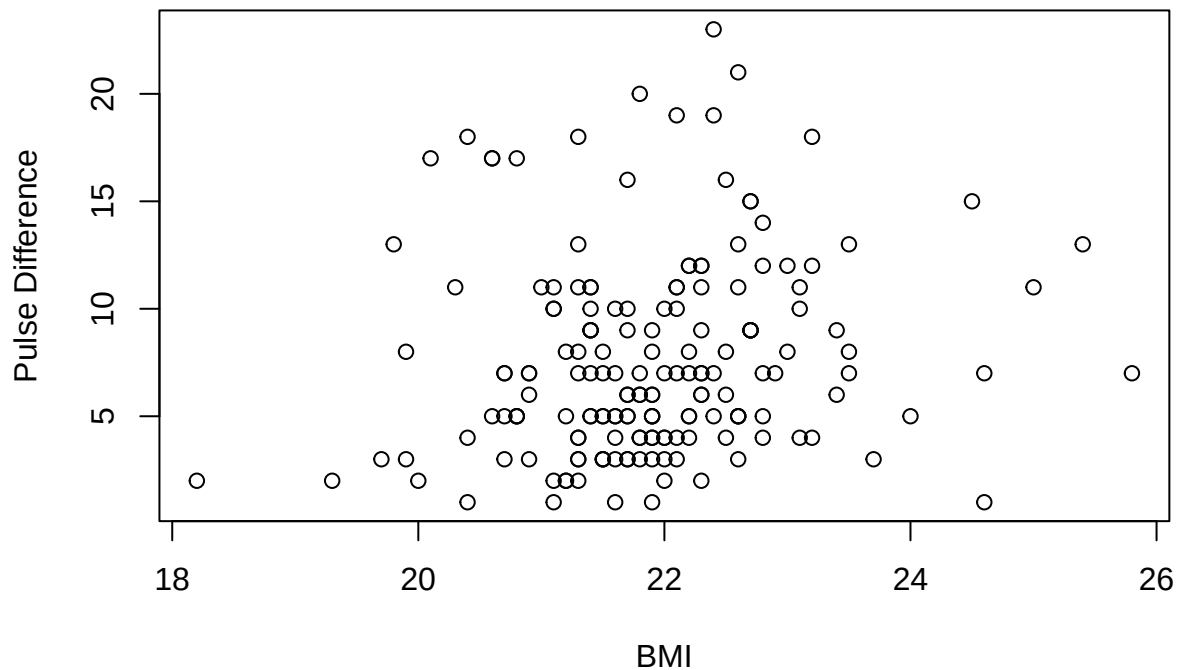
```
# Box plot for Stimulated Pulse by Treatment
```

```
boxplot(stimulated_pulse ~ treatment, main = "Stimulated Pulse",
        xlab = "Treatment", ylab = "Stimulated Pulse")
```



```
# Plot the pulse difference
```

```
plot(pulse_diff ~ bmi, xlab = "BMI", ylab = "Pulse Difference")
```



```
remove_outliers <- function(df, column_name) {
  Q1 <- quantile(df[[column_name]], 0.25)
  Q3 <- quantile(df[[column_name]], 0.75)
  IQR <- Q3 - Q1
  lower_bound <- Q1 - 1.5 * IQR
  upper_bound <- Q3 + 1.5 * IQR
  df <- df[df[[column_name]] >= lower_bound & df[[column_name]] <= upper_bound, ]
  return(df)
}

# Remove the outliers
# dat <- remove_outliers(dat, "bmi")
# dat <- remove_outliers(dat, "rest_pulse")
# dat <- remove_outliers(dat, "stimulated_pulse")
# Attach the data frame
# attach(dat)

# Set up the plot area
par(mfrow = c(2, 2))

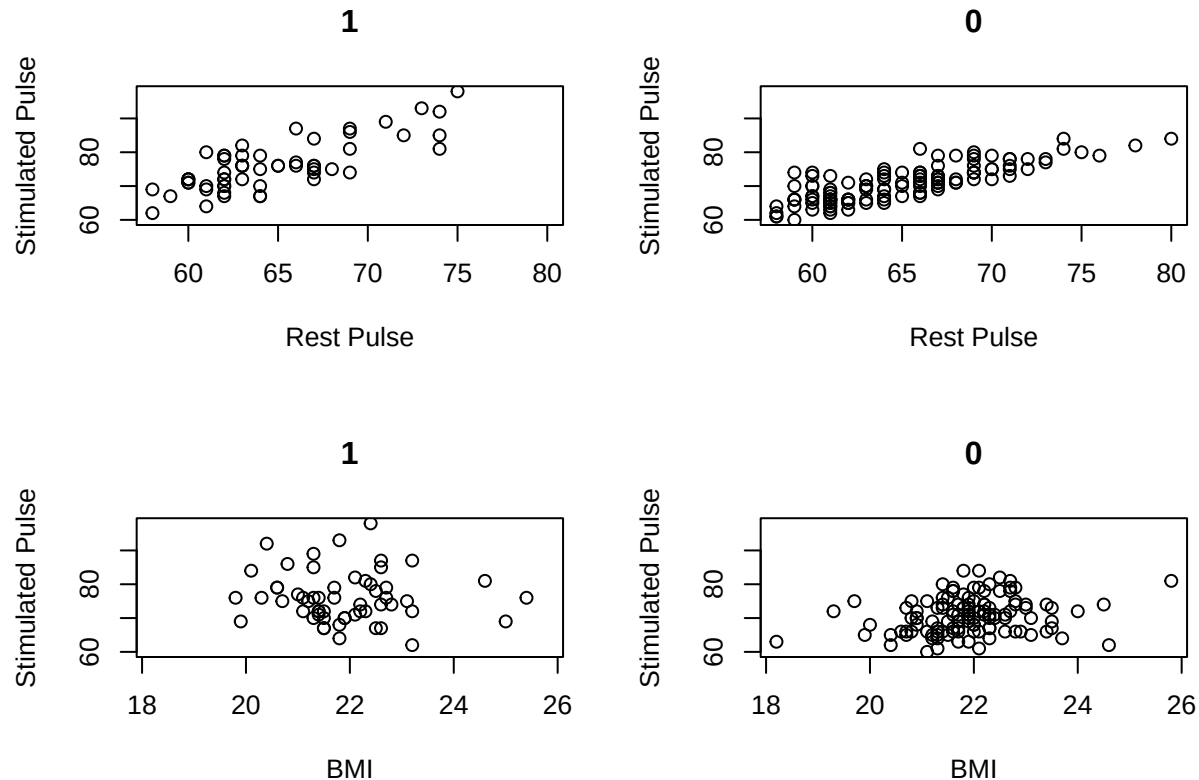
# Scatter plots for the rest pulse
plot(rest_pulse[treatment == 0], stimulated_pulse[treatment == 0],
     xlab = "Rest Pulse", ylab = "Stimulated Pulse",
     main = 1, xlim = range(rest_pulse), ylim = range(stimulated_pulse))
plot(rest_pulse[treatment == 1], stimulated_pulse[treatment == 1],
     xlab = "Rest Pulse", ylab = "Stimulated Pulse",
```

```

    main = 0, xlim = range(rest_pulse), ylim = range(stimulated_pulse))

# Scatter plots for the BMI
plot(bmi[treatment == 0], stimulated_pulse[treatment == 0],
     xlab = "BMI", ylab = "Stimulated Pulse",
     main = 1, xlim = range(bmi), ylim = range(stimulated_pulse))
plot(bmi[treatment == 1], stimulated_pulse[treatment == 1],
     xlab = "BMI", ylab = "Stimulated Pulse",
     main = 0, xlim = range(bmi), ylim = range(stimulated_pulse))

```



3 Initial model suggested by the clinician

```

# Fit the initial linear model
fit0 <- lm(stimulated_pulse ~ rest_pulse + treatment)

# Set up the plot area
par(mfrow = c(2, 2))

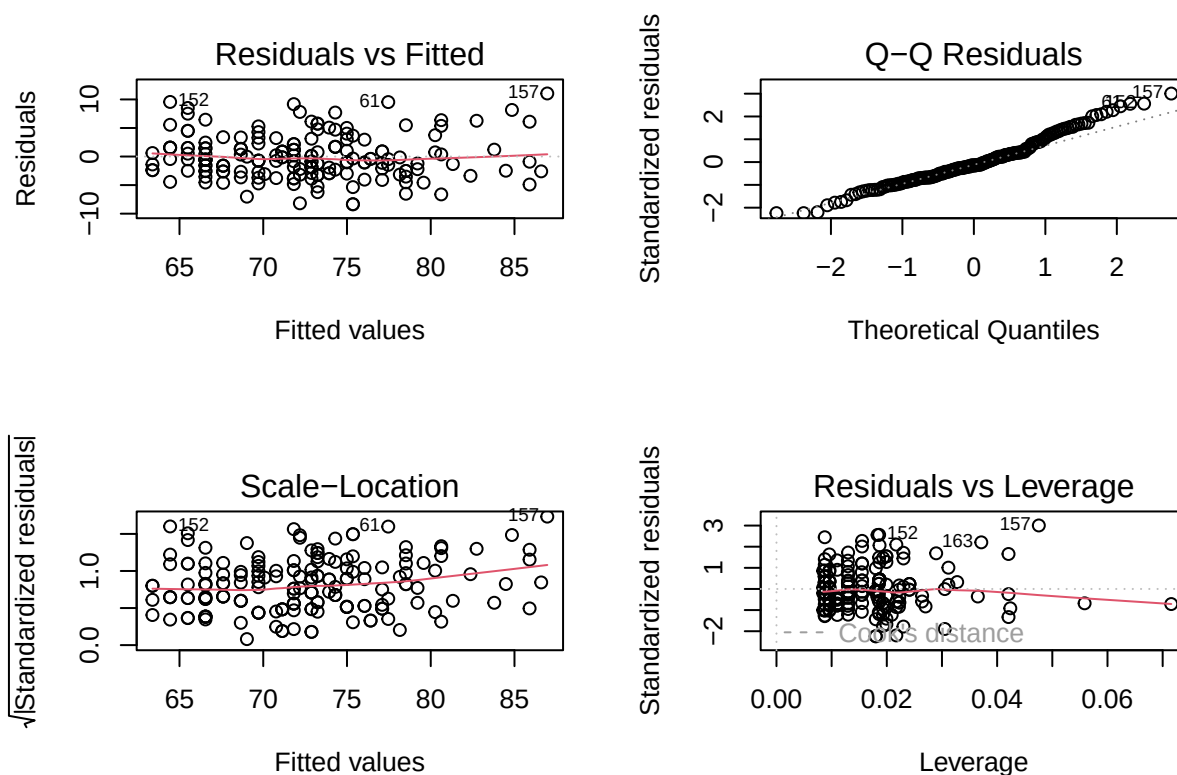
# Summary of the initial model
summary(fit0)

##
## Call:
## lm(formula = stimulated_pulse ~ rest_pulse + treatment)
##

```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.3537 -2.5430 -0.5512  1.6656 11.0426
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   7.8412     4.1143   1.906  0.0584 .
## rest_pulse    1.0549     0.0631  16.718 <2e-16 ***
## treatment1   -5.6378     0.6114  -9.221 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.766 on 171 degrees of freedom
## Multiple R-squared:  0.6752, Adjusted R-squared:  0.6714
## F-statistic: 177.7 on 2 and 171 DF,  p-value: < 2.2e-16

# Plot the initial model
par(mfrow = c(2, 2))
plot(fit0)
```



4 Model suggested by the statistician

```
custom_glm_model <-
  function(X,
    y,
```

```

        max_iter = 100,
        tol = 1e-4,
        eps = 1e-6) {
  # Initialize mu and eta
  mu <- rep(mean(y), length(y))
  eta <- 1 / mu

  # Initialize beta
  beta <- rep(0, ncol(X))

  for (i in 1:max_iter) {
    # Update the adjusted dependent variable z
    z <- eta - ((y - mu) / mu ^ 2)

    # Update the weights
    w <- mu ^ 2

    # Update the model using a weighted least squares fit
    WLS_fit <- lm(z ~ bmi + treatment, weights = w)
    new_beta <- as.numeric(WLS_fit$coeff)

    # Check if the coefficients have converged
    if (max(abs(new_beta - beta)) < tol) {
      return(new_beta)
    }

    # Update the coefficients
    beta <- new_beta

    # Update the linear predictor
    eta <- X %*% beta

    # Update the mean
    mu <- 1 / eta
  }

  return (beta)
}

X <- cbind(1, bmi, treatment)
y <- pulse_diff

# Fit the custom GLM model
coeffs <- custom_glm_model(X, y, max_iter=1000)
coeffs

```

```
## [1] -0.049360490  0.004422611  0.094761065
```

The algorithm converges pretty rapidly and we get similar coefficients to the ones obtained by the R's `glm` function. Hence, from now on, we will use the `glm` function to fit the models.

```

# Fit the GLM model
fit1_form <- (pulse_diff ~ bmi + treatment)
fit1 <- glm(fit1_form, family = Gamma(link = "inverse"), data = dat)

```

```
# Summary of the GLM model
```

```
summary(fit1)
```

```
##
```

```
## Call:
```

```
## glm(formula = fit1_form, family = Gamma(link = "inverse"), data = dat)
```

```
##
```

```
## Coefficients:
```

```
##           Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)  0.290659   0.085576   3.397 0.000849 ***
```

```
## bmi         -0.009202   0.003843  -2.395 0.017707 *
```

```
## treatment1  0.084694   0.009740   8.695 2.76e-15 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## (Dispersion parameter for Gamma family taken to be 0.2425243)
```

```
##
```

```
## Null deviance: 67.433  on 173  degrees of freedom
```

```
## Residual deviance: 47.430  on 171  degrees of freedom
```

```
## AIC: 920.98
```

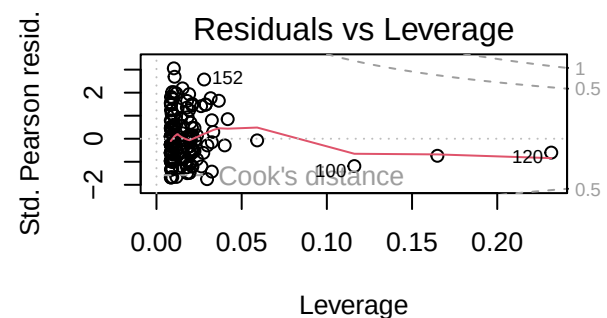
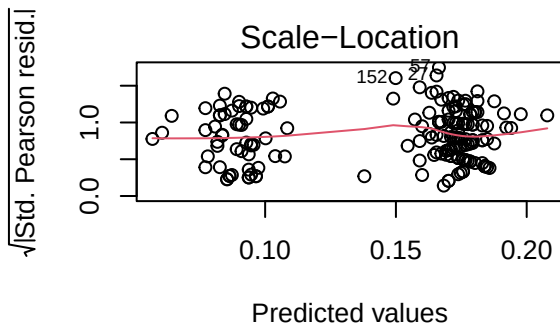
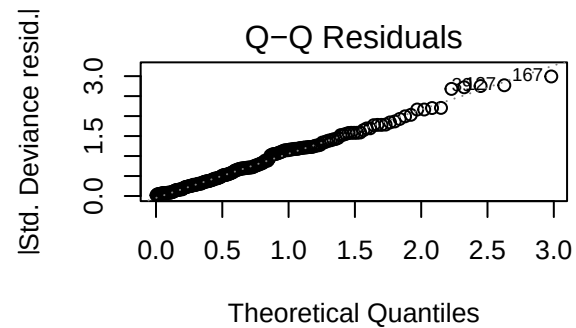
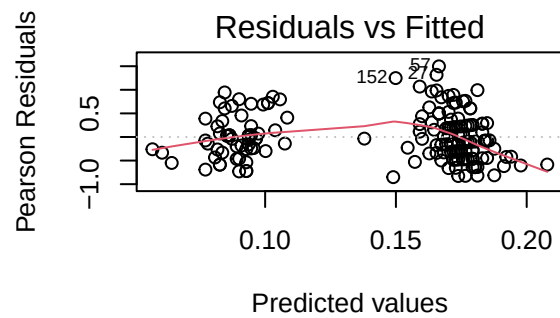
```
##
```

```
## Number of Fisher Scoring iterations: 5
```

```
# Plot the GLM model
```

```
par(mfrow = c(2, 2))
```

```
plot(fit1)
```



5 Estimation of the treatment effect

```
ci_bootstrap <- function(data, fit_form, num_iterations = 1000, seed=60012) {  
  # Set the seed for reproducibility  
  set.seed(seed)  
  
  # Compute the typical BMI (mean BMI for the sample)  
  typical_bmi <- mean(bmi)  
  
  # Extract the number of points in the data  
  num_points <- nrow(dat)  
  
  # Create an empty vector to store the predicted differences  
  predicted_diff <- numeric(num_iterations)  
  
  for (i in 1:num_iterations) {  
    # Sample the data with replacement  
    indexes <- sample(num_points, num_points, replace = TRUE)  
  
    # Train the model  
    fit_i <- glm(fit_form, family = Gamma(link = "inverse"), data = data[indexes,])  
  
    # Predict the difference in pulse  
    new_data <- data.frame(bmi = typical_bmi,  
                          treatment = factor(c(0, 1), levels = levels(treatment)))  
  
    predictions <- predict(fit_i, newdata = new_data, type = "response")  
    predicted_diff[i] <- predictions[1] - predictions[2]  
  }  
  
  # Compute the estimated difference  
  estimated_diff <- mean(predicted_diff)  
  
  # Compute the 95% confidence interval  
  conf_interval <- quantile(predicted_diff, c(0.025, 0.975))  
  
  return(list(estimated_diff = estimated_diff,  
             conf_interval = conf_interval))  
}  
  
ci_bootstrap(dat, fit1_form, num_iterations = 1000)  
  
## $estimated_diff  
## [1] 5.457723  
##  
## $conf_interval  
##      2.5%      97.5%  
## 4.029513 6.923186
```

6 Changing the linear predictors

```
# Fit the GLM model  
fit2 <- glm(pulse_diff ~ bmi, family = Gamma(link = "inverse"), data = dat)
```



```
# Summary of the GLM model
```

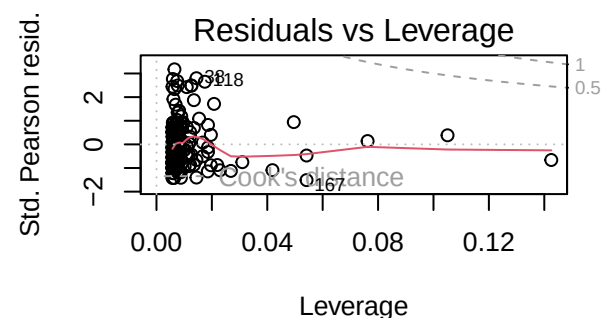
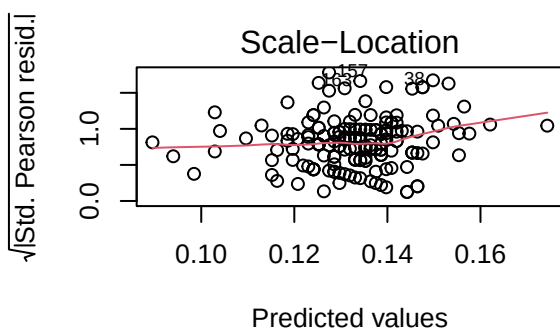
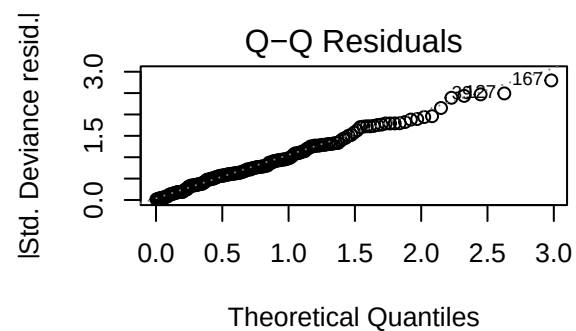
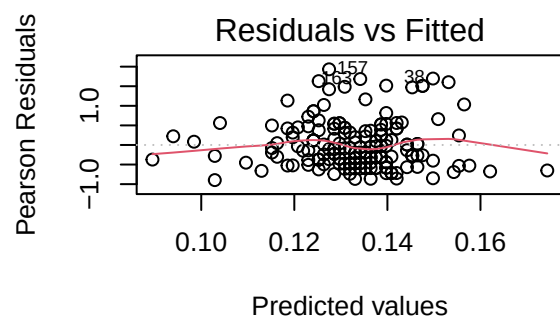
```
summary(fit2)
```

```
##
## Call:
## glm(formula = pulse_diff ~ bmi, family = Gamma(link = "inverse"),
##      data = dat)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.377514   0.118314   3.191  0.00169 **
## bmi         -0.011163   0.005344  -2.089  0.03818 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Gamma family taken to be 0.3723593)
##
## Null deviance: 67.433  on 173  degrees of freedom
## Residual deviance: 65.909  on 172  degrees of freedom
## AIC: 979.28
##
## Number of Fisher Scoring iterations: 5
```

```
# Plot the GLM model
```

```
par(mfrow = c(2, 2))
```

```
plot(fit2)
```



```

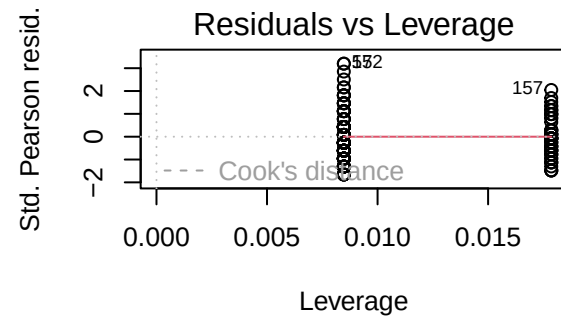
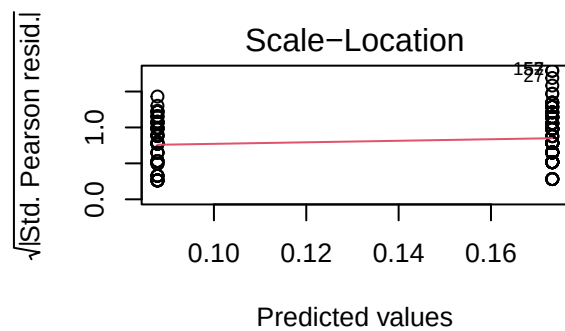
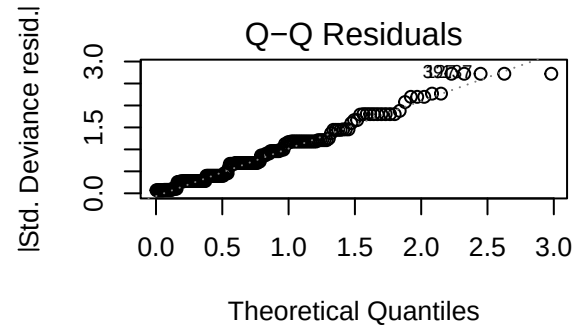
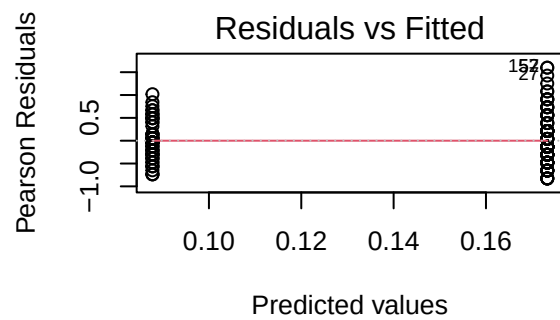
# Fit the GLM model
fit3 <- glm(pulse_diff ~ treatment, family = Gamma(link = "inverse"), data = dat)

# Summary of the GLM model
summary(fit3)

##
## Call:
## glm(formula = pulse_diff ~ treatment, family = Gamma(link = "inverse"),
##      data = dat)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.087774    0.005891  14.900 < 2e-16 ***
## treatment1  0.085500    0.009944   8.598 4.83e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Gamma family taken to be 0.2522337)
##
##      Null deviance: 67.433  on 173  degrees of freedom
## Residual deviance: 48.707  on 172  degrees of freedom
## AIC: 923.81
##
## Number of Fisher Scoring iterations: 5

# Plot the GLM model
par(mfrow = c(2, 2))
plot(fit3)

```

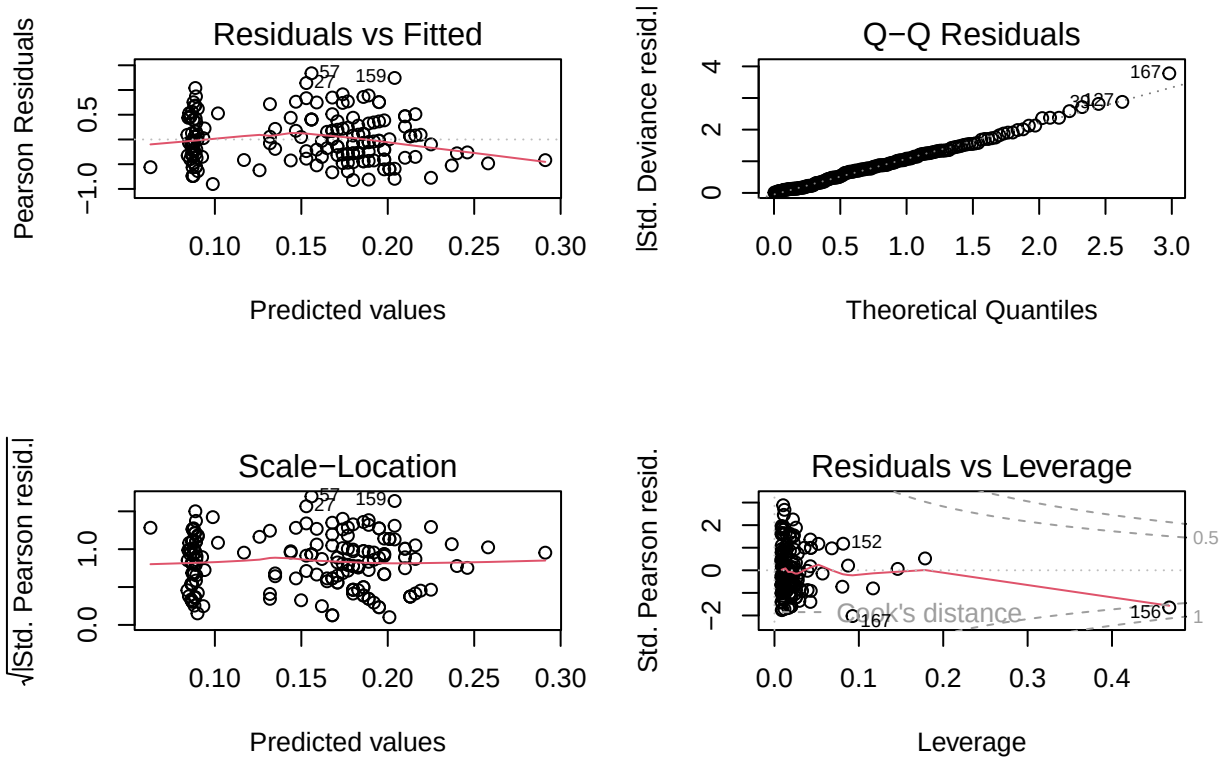


```
# Fit the GLM model
fit4_form <- (pulse_diff ~ bmi * treatment)
fit4 <- glm(fit4_form, family = Gamma(link = "inverse"), data = dat)

summary(fit4)

##
## Call:
## glm(formula = fit4_form, family = Gamma(link = "inverse"), data = dat)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.049042  0.109212   0.449   0.654
## bmi          0.001771  0.004992   0.355   0.723
## treatment1   0.788984  0.165805   4.759 4.15e-06 ***
## bmi:treatment1 -0.031821  0.007466 -4.262 3.35e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Gamma family taken to be 0.2181898)
##
##      Null deviance: 67.433  on 173  degrees of freedom
## Residual deviance: 43.796  on 170  degrees of freedom
## AIC: 908.51
##
## Number of Fisher Scoring iterations: 6
```

```
# Plot the GLM model
par(mfrow = c(2, 2))
plot(fit4)
```



Now let's compare the models.

```
# Compare the models
anova(fit1, fit2, fit3, fit4)
```

```
## Analysis of Deviance Table
##
## Model 1: pulse_diff ~ bmi + treatment
## Model 2: pulse_diff ~ bmi
## Model 3: pulse_diff ~ treatment
## Model 4: pulse_diff ~ bmi * treatment
##   Resid. Df Resid. Dev Df Deviance
## 1      171      47.430
## 2      172      65.909 -1 -18.4786
## 3      172      48.707  0  17.2021
## 4      170      43.796  2   4.9109
```

```
AIC(fit1, fit2, fit3, fit4)
```

```
##      df      AIC
## fit1  4 920.9784
## fit2  3 979.2752
## fit3  3 923.8108
## fit4  5 908.5056
```

```

# Pick the best model
fit_form <- fit4_form
fit <- fit4

# Make confidence intervals for the coefficients
confint(fit, level = 0.95)

## Waiting for profiling to be done...

##              2.5 %      97.5 %
## (Intercept) -0.168640884  0.25701541
## bmi         -0.007673459  0.01178051
## treatment1  0.456602645  1.10559051
## bmi:treatment1 -0.046078094 -0.01684352

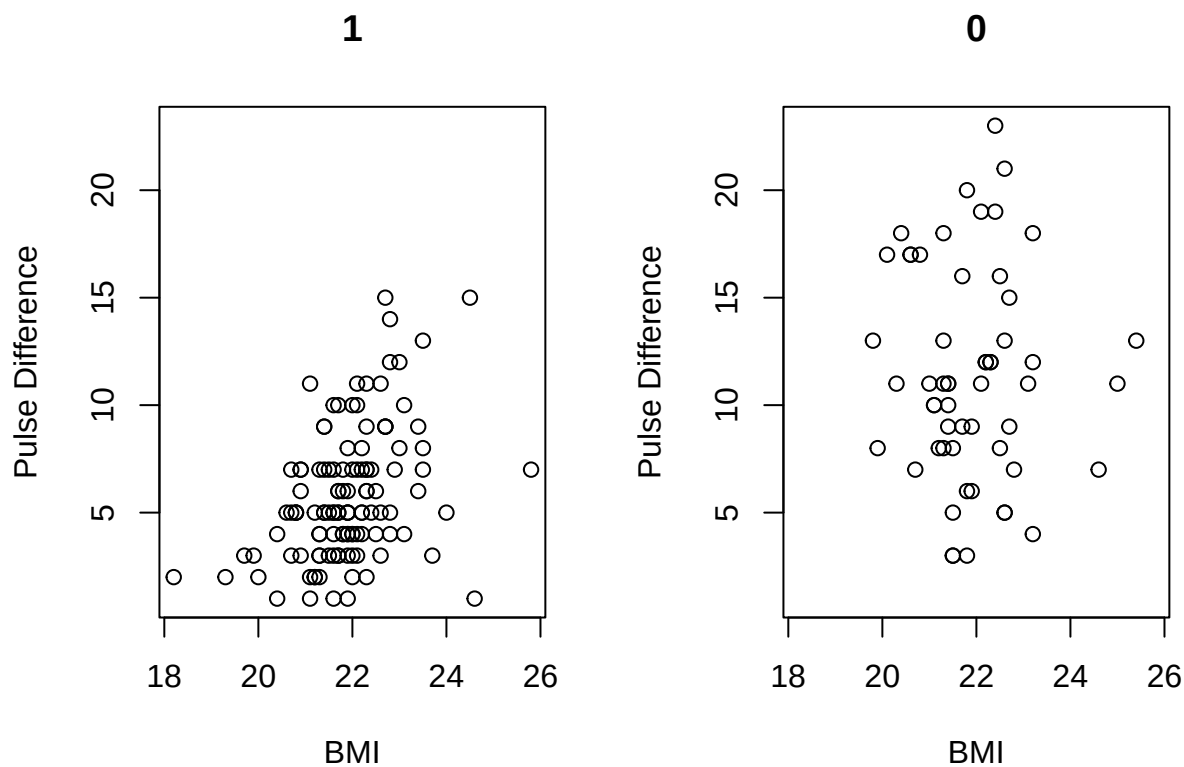
```

7 Expected increase in heart rate

```

# Plot the difference in heart rate as a function of BMI
par(mfrow = c(1, 2))
plot(bmi[treatment == 1], pulse_diff[treatment == 1],
     xlab = "BMI", ylab = "Pulse Difference",
     main = 1, xlim = range(bmi), ylim = range(pulse_diff))
plot(bmi[treatment == 0], pulse_diff[treatment == 0],
     xlab = "BMI", ylab = "Pulse Difference",
     main = 0, xlim = range(bmi), ylim = range(pulse_diff))

```



```

make_predictions <- function(fit, range) {
  # Prepare a data frame for prediction: repeat each BMI for both treatment levels
  data_stimulant = data.frame(bmi = range, treatment = factor(0, levels = levels(treatment)))
  data_placebo = data.frame(bmi = range, treatment = factor(1, levels = levels(treatment)))

  # Predict heart rate increase for the new data
  predictions_stimulant = predict(fit, newdata = data_stimulant, type = "response")
  predictions_placebo = predict(fit, newdata = data_placebo, type = "response")

  # Combine the prediction data frames into one,
  # adding a column to distinguish between stimulant and placebo
  data_combined <- rbind(data_stimulant, data_placebo)
  data_combined$treatment <- factor(data_combined$treatment, labels = c(0, 1))

  # Add the predictions as a new column in the combined data frame
  data_combined$expected_increase <- c(predictions_stimulant, predictions_placebo)

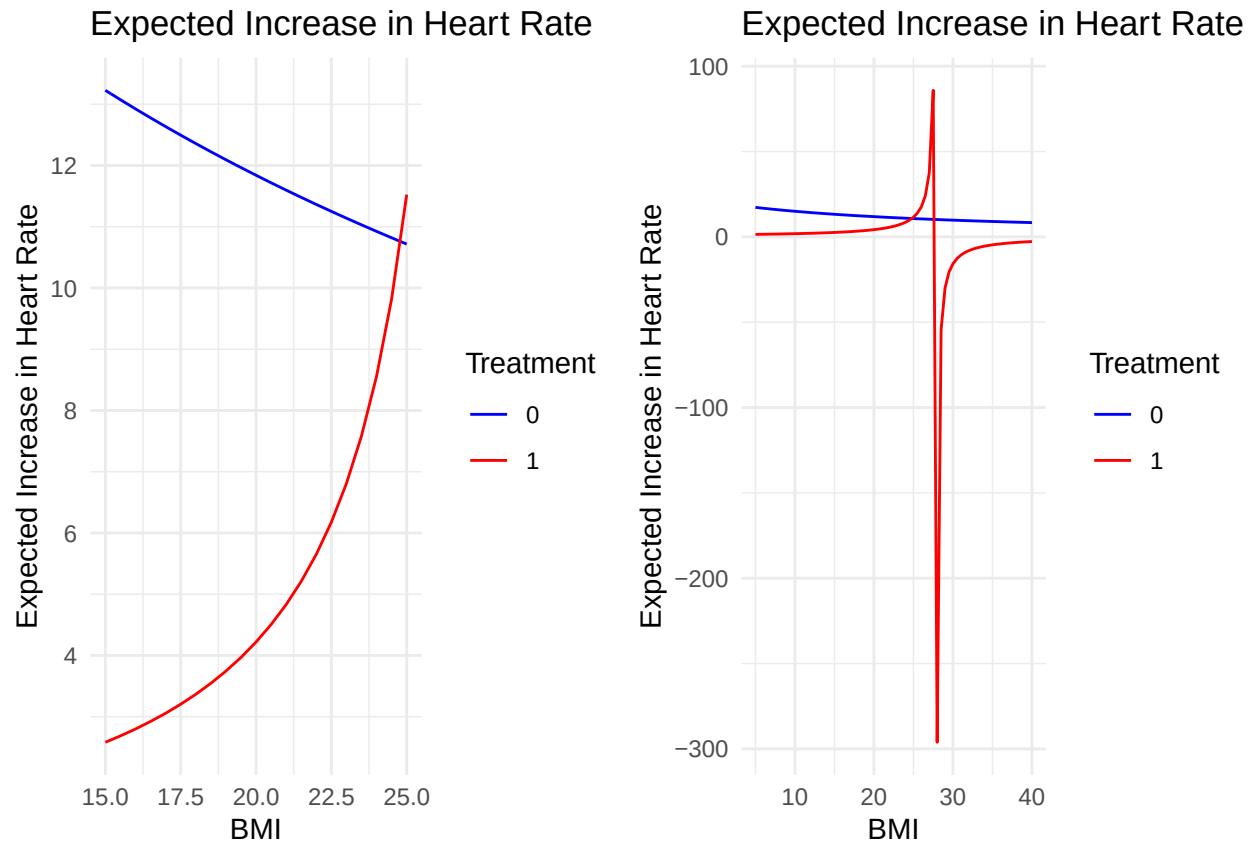
  return (data_combined)
}

plot_predictions <- function(data_combined) {
  # Use ggplot2 to plot the expected increase in heart rate as a function of BMI
  return (ggplot(data_combined, aes(x = bmi, y = expected_increase, color = treatment)) +
    geom_line() +
    labs(title = "Expected Increase in Heart Rate",
         x = "BMI",
         y = "Expected Increase in Heart Rate") +
    scale_color_manual(values = c("blue", "red")) +
    theme_minimal() +
    guides(color = guide_legend(title = "Treatment")))
}

# Make predictions
predictions_1 <- make_predictions(fit, range = seq(from = 15, to = 25, by=0.5))
predictions_2 <- make_predictions(fit, range = seq(from = 5, to = 40, by=0.5))

# Plot these values on the same graph
grid.arrange(plot_predictions(predictions_1), plot_predictions(predictions_2), ncol = 2)

```



8 Simulation study to determine the size of treatment effect

```
# Set the seed
set.seed(5039)

num_simulations <- 600
treatment_effects <- seq(from = 0.1, to = 20, by = 0.1)

# Create an empty vector to store the results
power <- numeric(length(treatment_effects))

# Run the simulations
for (i in seq_along(treatment_effects)) {
  effect_size <- treatment_effects[i]
  insignificant_count <- 0

  # Run the simulations for the current effect size
  for (j in 1:num_simulations) {
    # Create a copy of the dataset for manipulation
    sim_data <- dat

    # Simulate the response variable
    sim_data$pulse_diff_sim <- unlist(simulate(fit))

    # Update the response variable for the placebo group
```

```

# (we're bringing them to the same level as the stimulant group)
# Notice that we're adding the treatment effect to the placebo group because
# if we would subtract it from the stimulant group,
# we would encounter errors with the Gamma model fitting.
indexes <- sim_data$treatment == 1
sim_data$pulse_diff_sim[indexes] <- sim_data$pulse_diff_sim[indexes] + effect_size

# Fit the model to the simulated data
form = pulse_diff_sim ~ bmi * treatment
fit_sim <- glm(form, family = Gamma(link = "inverse"), data = sim_data)

treatment_is_insignificant <- summary(fit_sim)$coefficients["treatment1", "Pr(>|t|)"] > 0.05
bmi_treatment_interaction_is_insignificant <- summary(fit_sim)$coefficients["bmi:treatment1", "Pr(>|t|)"] > 0.05

# Check if the treatment effect is insignificant
if (treatment_is_insignificant && bmi_treatment_interaction_is_insignificant) {
  insignificant_count <- insignificant_count + 1
}
}

# Store the power for the current effect size
power[i] <- insignificant_count / num_simulations
}

# Print the treatment effect after which the power is greater than 0.05
treatment_effects[which(power > 0.05)[1:10]]

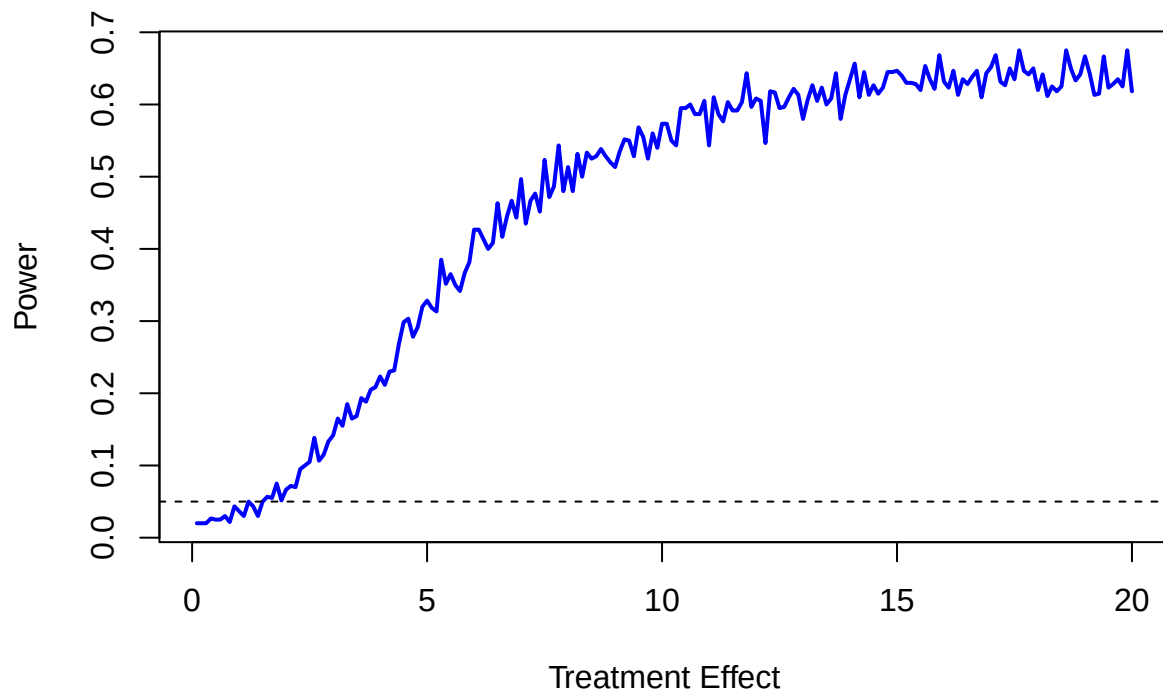
## [1] 1.6 1.7 1.8 1.9 2.0 2.1 2.2 2.3 2.4 2.5

# Print the treatment effect after which the power is greater than 0.6
treatment_effects[which(power > 0.6)[1:10]]

## [1] 10.9 11.1 11.4 11.7 11.8 12.0 12.1 12.3 12.4 12.7

# Plot the power
par(mfrow = c(1, 1))
plot(treatment_effects, power, type = "l", xlab = "Treatment Effect", ylab = "Power", col = "blue", lwd = 2)
abline(h = 0.05, lty = 2) # Add a horizontal line at 0.05

```

9 Simulation to assess the suitability of the commonly used asymptotic approximations for the sampling distribution of the maximum likelihood estimators and scaled deviance in this problem

```
# Set the seed for reproducibility
set.seed(604859)

# Pick the model
model <- fit

# Set the number of simulations
num_sim <- 1000

# Extract some useful quantities
num_subjects <- nrow(dat)
num_coeffs = model$rank
dof = num_subjects - num_coeffs
coeff_names <- rownames(summary(model)$coefficients)

# Create empty matrices to store the results
coeffs_sim <- matrix(nrow = num_sim, ncol=num_coeffs)
dev_sim <- numeric(num_sim)
```

```

# Run the simulations
for (i in 1:num_sim) {
  # Simulate the output variable
  diff_sim <- unlist(simulate(model))

  # Fit the model to the simulated data
  form = diff_sim ~ bmi * treatment
  fit_sim <- glm(form, family = Gamma(link = "inverse"), data = dat)
  summary_fit_sim <- summary(fit_sim)

  # Get the dispersion parameter
  dispersion <- summary_fit_sim$dispersion

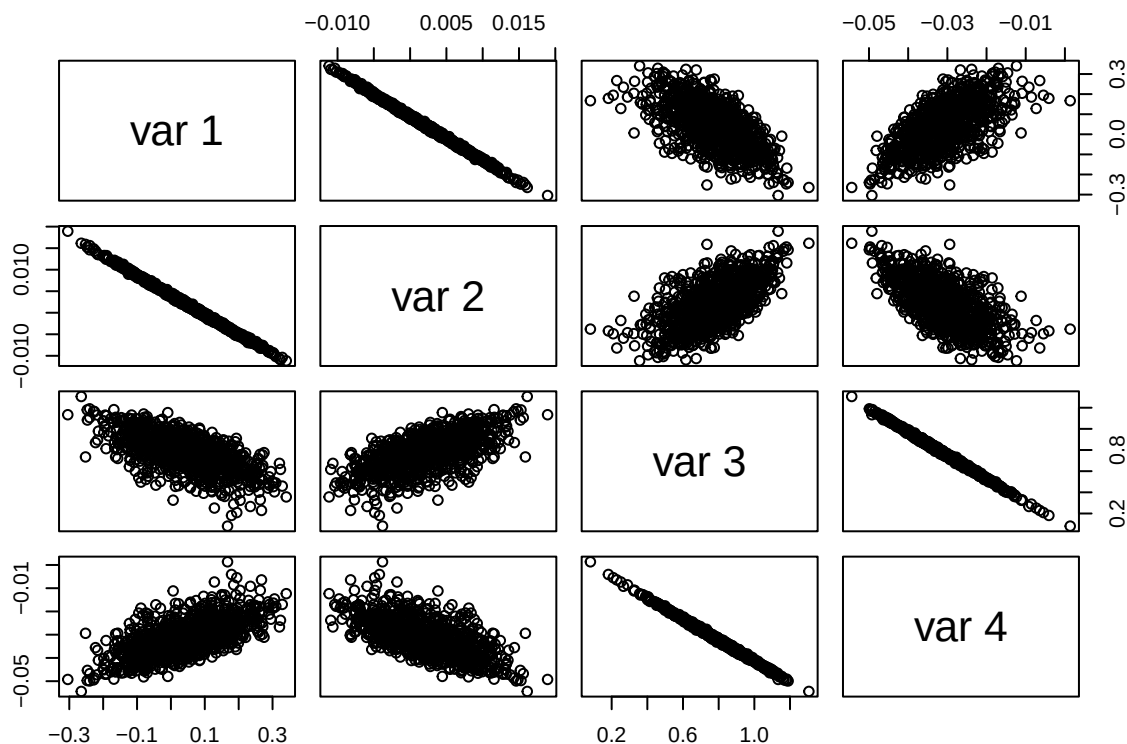
  # Store the coefficients
  coeffs_sim[i,] <- coef(fit_sim)
  # Store the scaled deviance
  dev_sim[i] <- summary_fit_sim$deviance / dispersion
}

```

```

# Plot the pairs of coefficients
pairs(coeffs_sim)

```



```

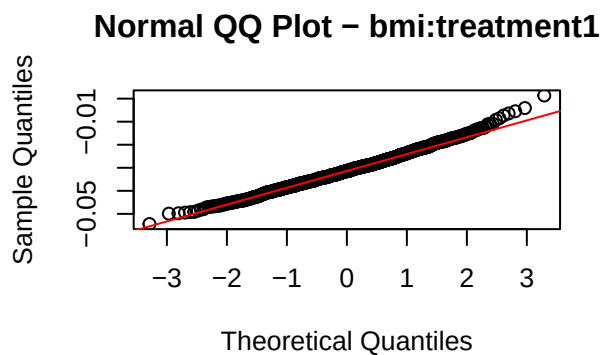
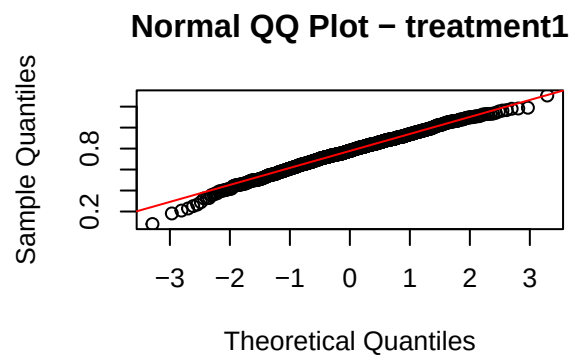
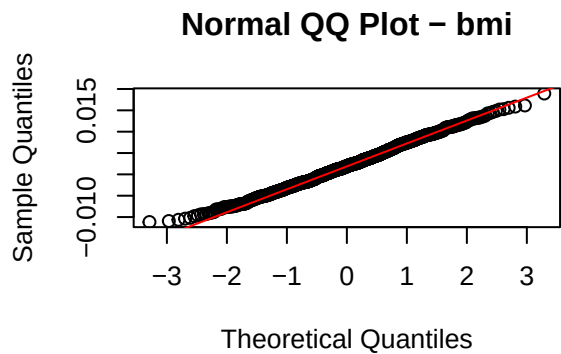
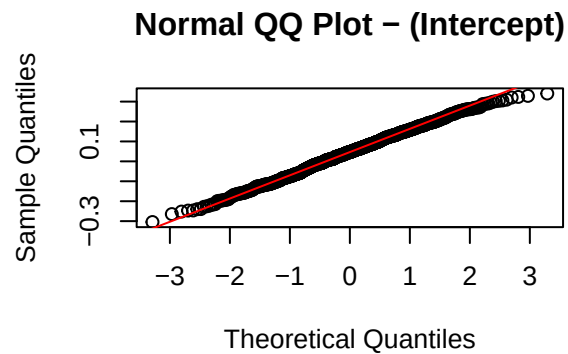
# Compare the residuals to the normal distribution
par(mfrow = c(2,ifelse(num_coeffs %% 2 == 0, num_coeffs / 2, (num_coeffs + 1) / 2)))
for(j in 1:num_coeffs){
  qqnorm(coeffs_sim[,j], main=paste("Normal QQ Plot -", coeff_names[j]))
}

```

```

  qqline(coeffs_sim[,j], col = "red")
}

```



```

# Perform the Kolmogorov-Smirnov test for the residuals
for(j in 1:num_coeffs){
  standardized_residuals <- (coeffs_sim[,j] - mean(coeffs_sim[,j])) / sd(coeffs_sim[,j])
  print(paste("Kolmogorov-Smirnov test p-value for coefficients",
    coeff_names[j], ": ",
    ks.test(standardized_residuals, "pnorm")$p.value))
}

```

```

## [1] "Kolmogorov-Smirnov test p-value for coefficients (Intercept) : 0.741003637019485"
## [1] "Kolmogorov-Smirnov test p-value for coefficients bmi : 0.69453565351338"
## [1] "Kolmogorov-Smirnov test p-value for coefficients treatment1 : 0.319987566817416"
## [1] "Kolmogorov-Smirnov test p-value for coefficients bmi:treatment1 : 0.277589941502756"

```

```

# Perform a test from the MVN package in order to check the multivariate normality
mvn(coeffs_sim, mvnTest = "mardia", desc=TRUE)$multivariateNormality

```

```

##           Test           Statistic           p value Result
## 1 Mardia Skewness 191.060217749739 6.54281112386184e-30    NO
## 2 Mardia Kurtosis  3.86643419308828 0.000110438245979338    NO
## 3           MVN           <NA>           <NA>           NO

```

```

# Plot the histogram of simulated deviances
par(mfrow = c(1, 1))
hist(dev_sim, breaks=40, probability = TRUE,
  main="Histogram of Simulated Scaled Deviances",

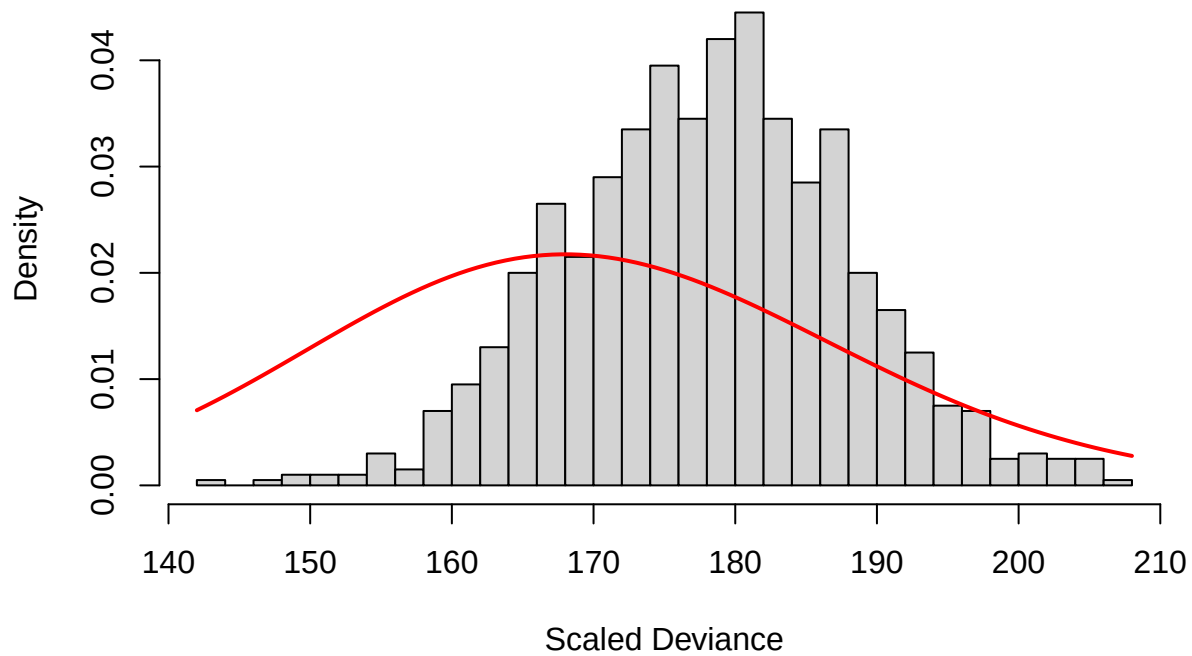
```

```

xlab="Scaled Deviance")
# Overlay the chi-squared distribution
curve(dchisq(x, df=dof), add=TRUE, col="red", lwd=2)

```

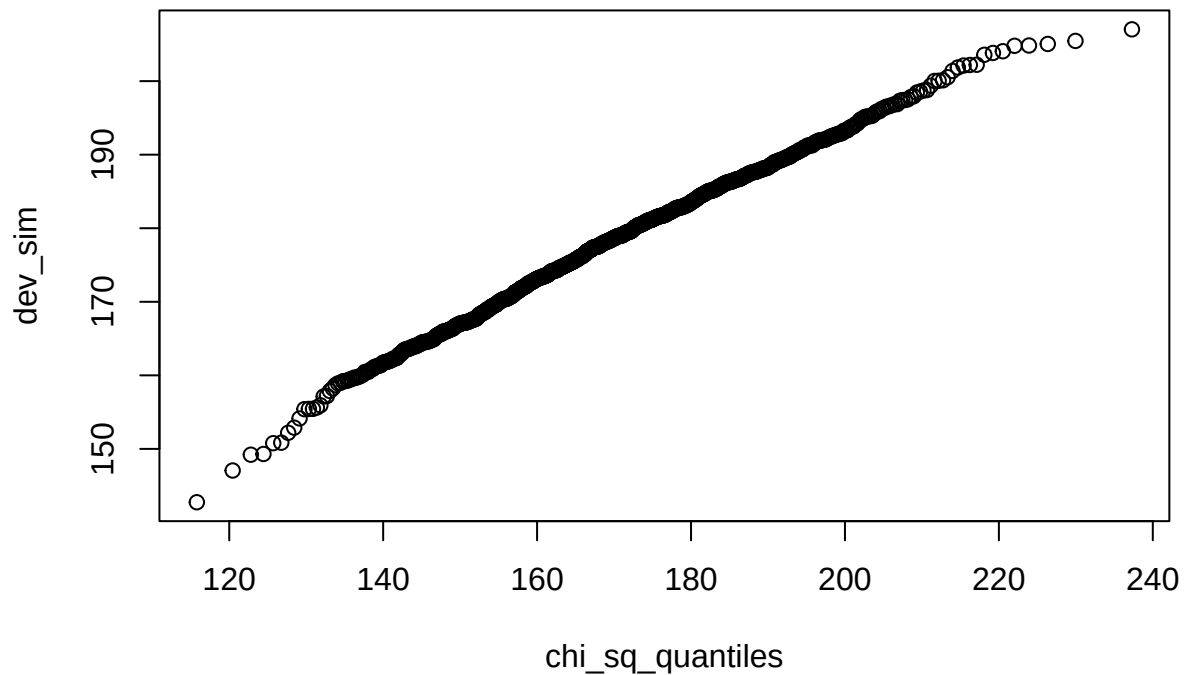
Histogram of Simulated Scaled Deviances



```

# Compare the scaled deviance to the chi-squared distribution
par(mfrow = c(1, 1))
chi_sq_quantiles <- qchisq(ppoints(dev_sim), df = dof)
qqplot(chi_sq_quantiles, dev_sim)
qqline(dev_sim, col = "red")

```



```
# Perform a Kolmogorov-Smirnov test
ks.test(dev_sim, "pchisq", dof)

##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: dev_sim
## D = 0.31687, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

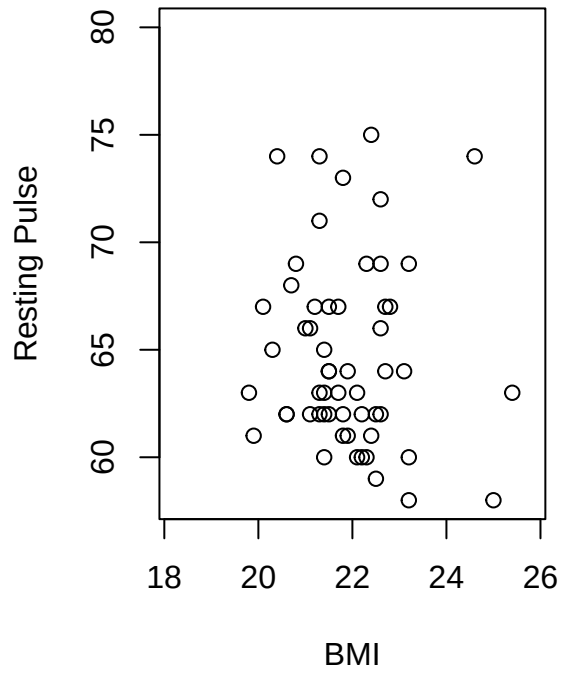
10 Relationship between BMI and resting pulse

```
# Print the correlation between BMI and resting pulse
cor(bmi, rest_pulse)

## [1] -0.02819013

# Print the resting pulse as a function of BMI
par(mfrow = c(1, 2))
plot(bmi[treatment == 0], rest_pulse[treatment == 0],
     xlab = "BMI", ylab = "Resting Pulse",
     main = "Treatment", xlim = range(bmi), ylim = range(rest_pulse))
plot(bmi[treatment == 1], rest_pulse[treatment == 1],
     xlab = "BMI", ylab = "Resting Pulse",
     main = "Placebo", xlim = range(bmi), ylim = range(rest_pulse))
```

Treatment



Placebo

